NPTEL Software Testing - Simple Explanations

Absolutely. Since your **NPTEL Software Testing exam is tomorrow**, let's make this easy to learn rather than textbook-heavy. I'll explain every major topic using the same pattern:

Idea → Simple example → What to remember → Exam trap

1. Fault, Error and Failure

These three sound similar, but they mean different things.

Fault

A **fault** is the actual mistake in the program.

Suppose the requirement is:

> A person is eligible if age is **18 or above**.

But the programmer writes:

```
if (age > 18)
    eligible = true;
```

The programmer should have written `age >= 18`.

That wrong operator is the **fault**.

Error

When we execute with:

```
age = 18
```

the program calculates:

```
eligible = false
```

The program's **internal state is now wrong**. That's an error.

Failure

The screen displays:

```
Not eligible
```

Now the incorrect state has reached the user. That's a **failure**.

Memory trick

> **F → E → F**
> Fault → Error → Failure
> **Code wrong → State wrong → Output wrong**

---

2. RIPR Model

A fault doesn't automatically produce a visible failure.

Four things need to happen:

R: Reachability

The test needs to execute the faulty statement.

I: Infection

The fault must make the program state incorrect.

P: Propagation

The incorrect state must travel toward an observable output.

R: Revelation

Your test must actually check that output.

Example

Fault:

```
if (age > 18)
```

Test:

```
age = 25
```

The faulty code is reached, but both `>18` and `>=18` produce true.

So:

- Reachability ✅
- Infection ✅

The fault isn't detected.

Instead:

```
age = 18
```

Now the wrong `>` produces false whereas `>=` produces true.

Memory trick

> **RIPR = Reach → Infect → Propagate → Reveal**

Think:

> **Reach the bug, make it hurt, let it travel, observe it.**

---

3. Input Space Partitioning

Suppose a program accepts marks:

```
0-100
```

Testing every possible number is unnecessary.

Instead divide the inputs into **blocks**:

```
Block 1: marks < 0
Block 2: 0-100
Block 3: marks > 100
```

Then choose representative values:

```
-5
50
110
```

That’s Input Space Partitioning.

Characteristic

A characteristic describes **what aspect you’re dividing**.

Example: ATM withdrawal.

Characteristics could be:

```
Account balance
Withdrawal amount
Card status
PIN validity
```

Each has blocks.

For PIN:

```
Correct PIN
Incorrect PIN
```

---

4. Completeness and Disjointness

A good partition needs two properties.

Completeness

Every possible input belongs somewhere.

Suppose:

```
x < 0
x = 0
x > 0
```

Every number belongs to a block.

 Complete.

Memory

> **Complete = Nothing Missing**

Disjointness

An input should not belong to multiple blocks.

Bad example:

```
Block 1: x <= 10
Block 2: x >= 10
```

Where does `10` belong?

Both blocks.

 Not disjoint.

Better:

```
x <= 10
x > 10
```

Memory

> **Disjoint = No Overlap**

---

5. Interface-Based vs Functionality-Based ISP

Consider:

```
search(list, element)
```

Interface-based

Look separately at parameters.

For `list`:

```
null
empty
non-empty
```

For `element`:

```
null
non-null
```

Memory

> **Interface = Inputs individually**

Functionality-based

Think about what the function actually does.

For `search()`:

```
element absent
element occurs once
element occurs multiple times
```

This often captures behavior better.

Memory

> **Functionality = Function behavior**

---

6. Each Choice Coverage: ECC

Suppose:

```
Browser:
Chrome
Edge

OS:
Windows
Linux
Mac
```

ECC says:

> Every block must occur in at least one test.

Possible tests:

```
T1: Chrome + Windows
T2: Edge   + Linux
T3: Chrome + Mac
```

Every browser and OS choice appeared.

Memory

> **ECC = Each choice at least once**

---

7. All Combinations Coverage: ACoC

Use **every possible combination**.

For:

```
Browser = 2 choices
OS      = 3 choices
User    = 2 choices
```

Number of tests:

[2=12]

Formula

[ACoC=B_i]

where (B_i) is the number of blocks for characteristic (i).

Memory

> **ACoC = multiply everything**

---

8. Pair-Wise Coverage

Sometimes ACoC produces too many tests.

Pair-wise says:

> Make sure every possible combination of **two characteristics** occurs.

Example:

```
Browser
OS
User type
```

You ensure:

```
Browser × OS
Browser × User
OS × User
```

are covered.

You do **not necessarily test every three-way combination**.

Memory

> **Pair-wise = every handshake between 2 characteristics**

---

9. T-Wise Coverage

Pair-wise is simply:

[t=2]

3-wise means every combination involving three characteristics.

If there are `n` characteristics and:

[t=n]

then T-wise becomes **All Combinations Coverage**.

Memory

```
1-wise ≈ each choice
2-wise = pair-wise
3-wise = triples
...
n-wise = all combinations
```

---

10. Base Choice Coverage: BCC

Pick the most normal combination as the **base test**.

Example ATM:

```
Card = valid
PIN = correct
Balance = sufficient
```

That's your base.

Then change **one characteristic at a time**.

```
T1 valid   + correct   + sufficient
T2 expired + correct   + sufficient
T3 valid   + incorrect + sufficient
T4 valid   + correct   + insufficient
```

Formula

If characteristics have (B_i) blocks:

[BCC = 1+(B_i-1)]

Memory

> **BCC = Base + Change One**

---

11. Graph-Based Testing

Now instead of covering inputs, we're trying to cover the **structure of a program/ model**.

Imagine:

```
    1
    |
    2
   / \
  3   4
   \ /
    5
```

Numbers are **nodes**.

Connections are **edges**.

12. Node Coverage

Every reachable node must be executed.

For the above graph:

```
1 → 2 → 3 → 5
1 → 2 → 4 → 5
```

Together they cover:

```
1,2,3,4,5
```

So Node Coverage is satisfied.

Memory

> **Node Coverage = Visit every place**

---

13. Edge Coverage

Every reachable edge must be executed.

Suppose:

```
1 → 2
2 → 3
2 → 4
3 → 5
4 → 5
```

Every arrow has to be traversed.

Memory

> **Nodes = places**
>
> **Edges = roads**

Edge Coverage asks:

> Did I travel every road?

Generally:

[Edge Coverage Node Coverage]

---

14. Edge-Pair Coverage

Now we cover every path containing **two adjacent edges**.

For:

```
1 → 2 → 3
```

that's an edge pair:

```
(1,2),(2,3)
```

or node sequence:

```
[1,2,3]
```

Memory

> **Edge-Pair = two moves together**

---

15. Simple Path

A simple path doesn't repeat nodes, except that the first and last can be the same when representing a simple cycle.

Example:

```
1 → 2 → 3 → 4
```

Simple.

But:

```
1 → 2 → 3 → 2 → 4
```

is not simple because `2` repeats.

16. Prime Path

A **prime path** is a simple path that cannot be extended into a longer simple path.

Think of it as:

> **A maximal simple path**

Memory

> **Prime = longest possible simple route**

Prime Path Coverage requires every feasible prime path to be toured.

A useful hierarchy to remember is:

```
Node Coverage
     ↓
Edge Coverage
     ↓
Edge-Pair Coverage
     ↓
Prime Path Coverage
```

As you go down, the path obligations generally become stronger.

17. Control Flow Graph

A CFG represents program execution.

Example:

```
if (age >= 18)
    vote();
else
    reject();
```

Conceptually:

```
      condition
       /    \
     true   false
      |       |
    vote    reject
      \     /
        exit
```

For loops:

```
while (x > 0) {
    x--;
}
```

there is an edge returning to the loop condition.

Remember

- `if` → branch
- loop → branch + back edge
- `break` → loop exit
- `continue` → next iteration
- `return` → function exit

---

18. Data-Flow Testing

Graph testing asks:

> **Where does execution go?**

Data-flow testing asks:

> **Where does a value come from, and where is it used?**

Two important terms:

```
DEF
USE
```

---

19. Definition: DEF

When a variable receives a value:

```
x = 10;
```

That’s a **definition of x**.

---

20. USE

When we use the value:

```
y = x + 5;
```

x is being used.

There are two important types.

c-use

Computational use.

```
y = x + 5;
print(x);
```

Memory

> **C = Calculation**

---

p-use

Predicate use, meaning use inside a decision.

```
if (x > 5)
```

Memory

> **P = Predicate**

---

21. DU-Pair

A DU-pair connects:

```
Definition → Use
```

Example:

```
x = 10;        // definition

if (something) {
    ...
}

print(x);      // use
```

So we care whether the definition of x reaches the use.

---

22. Def-Clear Path

Consider:

```
x = 10;        // D1
...
x = 20;        // D2
...
print(x);      // U
```

The value from D1 does not reach U because x was redefined by D2.

Thus the path from D1 to U isn't **def-clear**.

A def-clear path means:

> No redefinition of that variable appears between its definition and use.

Memory

> **DEF → no new DEF → USE**

---

23. All-Defs Coverage

For **every definition**, reach at least **one** possible use.

Think:

> Every birth of a value must reach at least one consumer.

---

24. All-Uses Coverage

For each definition, cover **every reachable use**.

So:

```
Definition D1
   ↓
 Use U1
 Use U2
 Use U3
```

All three must be covered.

---

25. All-DU-Paths Coverage

Now cover **all simple def-clear paths** from every definition to its reachable uses.

Memory hierarchy:

```
All-Defs
   ↓
All-Uses
   ↓
All-DU-Paths
```

Memory trick

> **One → Every Use → Every Path**

---

26. Specification-Based Graph Testing

Graphs don’t have to come from source code.

They can come from:

- requirements
- state machines
- use cases
- workflows
- protocols

Imagine a turnstile.

States:

```
LOCKED
UNLOCKED
```

Events:

```
Coin
Push
```

Transitions:

```
LOCKED --coin--> UNLOCKED
UNLOCKED --push--> LOCKED
```

We can test whether every state and transition works correctly.

Memory

> **State = where you are**
>
> **Event = what happens**
>
> **Transition = where you go**

---

27. Logic Coverage

This is one of the **most important exam topics**.

Suppose:

```
if (age >= 18 && citizen == true)
```

The entire expression is the **predicate**:

[P=AB]

where:

```
A = age >= 18
B = citizen == true
```

A and B are **clauses**.

Memory

> Predicate = complete decision
> Clause = ingredient of decision

---

28. Predicate Coverage: PC

Make the overall predicate:

```
TRUE
FALSE
```

at least once each.

For:

[P=AB]

we could use:

```
A=T B=T → P=T
A=F B=T → P=F
```

PC achieved.

---

29. Clause Coverage: CC

Every individual clause must evaluate:

```
TRUE
FALSE
```

So both A and B need to become T and F.

Important:

> **Clause Coverage does not automatically imply Predicate Coverage in general.**

Likewise PC does not imply CC in general.

This is a common MCQ trap.

---

30. Combinatorial Coverage: CoC

Use **every truth-value combination**.

For two clauses:

```
A B
T T
T F
F T
F F
```

Total:

[2^2=4]

For three clauses:

[2^3=8]

For four:

[2^4=16]

Memory

> **n clauses → (2^n) combinations**

---

31. Major and Minor Clauses

Consider:

[P=AB]

Suppose we're examining A.

Then:

```
A = major clause
B = minor clause
```

We want to know:

> **Can changing A alone control the value of P?**

For:

[AB]

If:

```
B = true
```

then P is always true regardless of A.

Therefore A does **not** determine P.

But if:

```
B = false
```

then:

```
A=true  → P=true
A=false → P=false
```

Now A determines P.

Memory

> **Determination = major clause controls the decision**

---

32. GACC

General Active Clause Coverage

For every major clause:

- make it true and false
- it must determine the predicate
- minor clauses may differ between the paired tests

Memory

> **G = General, minors can change**

---

33. CACC

Correlated Active Clause Coverage

Same basic idea, but additionally:

> Predicate must change between the two tests.

Example:

```
Major A:

Test 1: A=T → P=T
Test 2: A=F → P=F
```

Memory

> **C = Correlated**
>
> Clause change correlates with predicate change.

---

34. RACC

Restricted Active Clause Coverage

The strongest restriction:

> Minor clauses must remain exactly the same.

Example:

[P=AB]

Choose A major.

```
Test 1:
A=T
B=F
P=T

Test 2:
A=F
B=F
P=F
```

Only A changed.

SUPER IMPORTANT MEMORY

```
GACC → minors may change

CACC → predicate must change

RACC → minors remain the same
```

Think:

> **G = General**
>
> **C = Correlated**
>
> **R = Restricted**

---

35. Inactive Clause Coverage

Active Clause Coverage asks:

> Can this clause control the predicate?

Inactive Clause Coverage asks:

> Can I create a situation where changing this clause **doesn't matter**?

For:

[P=AB]

When:

```
B=true
```

then:

```
A=true  → P=true
A=false → P=true
```

A is inactive.

---

36. Mutation Testing

Now deliberately introduce tiny bugs into the program.

Original:

```
if (x > 10)
```

Mutant:

```
if (x >= 10)
```

Now run the tests.

Suppose:

```
x = 10
```

Original:

```
false
```

Mutant:

```
true
```

Test detected the difference.

Therefore:

> **Mutant killed.**

Memory

> **Mutant killed = test caught the fake bug**

---

37. Mutation Operators

Typical mutations include:

Arithmetic

```
+ → -
* → /
```

Relational

```
> → >=
== → !=
```

Logical

```
&& → ||
```

Constant

```
10 → 0
```

Variable

```
x → y
```

Memory:

> **ARLCV**
>
> Arithmetic
> Relational
> Logical
> Constant
> Variable

---

38. Equivalent Mutant

Consider:

```
x = a + 0;
```

Mutant:

```
x = a;
```

For all inputs, the behavior is identical.

Therefore the mutant cannot be killed.

That’s an **equivalent mutant**.

Important

A surviving mutant doesn’t necessarily mean poor testing.

It might simply be equivalent.

---

39. Mutation Score

Conceptually:

[Mutation Score= {Total non-equivalent mutants}]

Example:

```
Total mutants = 100
Equivalent = 10

Non-equivalent = 90
Killed = 81
```

[Score==90%]

---

40. Weak vs Strong Mutation

Weak Mutation

Check the program state shortly after the mutated expression.

We mainly need:

```
Reachability
Infection
```

Strong Mutation

Difference must reach observable output.

We therefore care about propagation too.

Memory

> **Weak stops near the mutation**
>
> **Strong follows it to output**

---

41. Object-Oriented Testing

OO software introduces problems involving:

- inheritance
- overriding
- polymorphism
- dynamic binding
- object state

Example:

```
class Shape {
    area()
}

class Circle extends Shape {
    area()
}
```

Now:

```
Shape s = new Circle();
s.area();
```

Although variable `s` is declared as `Shape`, its runtime object is `Circle`.

Therefore the overridden:

```
Circle.area()
```

executes.

That's **dynamic binding**.

Memory

> **Declared type points. Runtime type chooses.**

---

42. Inheritance Testing

Suppose:

```
Vehicle
   ↓
 Car
```

`Car` inherits behavior from `Vehicle`.

We should test:

- inherited methods
- overridden methods
- changed state
- interactions between superclass and subclass

A method that worked with `Vehicle` may behave differently when executed on `Car`.

---

43. Polymorphism Testing

Suppose:

```
Shape s;
```

At runtime it might reference:

```
Circle
Square
Triangle
```

Tests should cover relevant runtime possibilities.

Don’t test only:

```
Shape → Circle
```

and assume every subtype behaves correctly.

---

44. Yo-Yo Graph

With deep inheritance and overriding, execution can effectively jump around the inheritance hierarchy.

Conceptually:

```
Parent
  ↓
Child
  ↑
Parent
  ↓
Child
```

Understanding where behavior comes from becomes difficult.

This is known as the **yo-yo effect**.

Memory

> **Yo-yo = control bouncing through inheritance levels**

---

45. Regression Testing

Suppose your application has 1,000 existing tests.

You fix one login bug.

Now the question is:

> Did the change break something that previously worked?

That’s regression testing.

---

46. Retest-All

Simply run:

```
All existing tests
```

Advantage

High confidence from the available suite.

Disadvantage

Can be expensive.

---

47. Regression Test Selection

Instead of 1,000 tests, determine which tests are affected by the change.

Suppose:

```
Changed:
Login

Dependencies:
Authentication
Session management
User profile
```

Select tests related to those affected areas.

Memory

> **Selection = Choose**

---

48. Test Minimization

Suppose:

```
100 tests
```

many cover the same obligations.

Minimization tries to remove redundant tests while retaining a chosen coverage target.

Risk

Two tests that cover the same statement may detect different bugs.

Memory

> **Minimization = Remove**

---

49. Test Prioritization

Don’t delete tests.

Just determine which should execute first.

For example:

```
1. Login tests
2. Authentication
3. Session
4. Checkout
5. Other tests
```

Memory trio

> **Selection = Choose**
>
> **Minimization = Remove**
>
> **Prioritization = Reorder**

Very useful MCQ distinction.

---

50. Non-Functional Testing

Functional testing asks:

> Does it do the correct thing?

Non-functional testing asks questions such as:

> How fast?
> How secure?
> How reliable?
> How scalable?

51. Load Testing

Test expected workload.

Suppose expected usage is:

```
500 simultaneous users
```

Test around that workload.

Memory

> **Load = Normal expected workload**

52. Stress Testing

Push beyond expected capacity.

```
500 expected

Test:
600
1000
2000
...
```

Observe where the system fails and how it fails.

Memory

> **Stress = Stretch until stressed**

53. Spike Testing

Sudden increase:

```
50 users
   ↓
1000 users suddenly
```

Check whether the system handles the sharp change.

Memory

> **Spike = Sudden**

---

54. Soak Testing

Run meaningful load for a long duration.

Useful for issues such as:

- resource leaks
- accumulated degradation
- long-running instability

Memory

> **Soak = Soak it for a long time**

The four are easy to memorize:

```
LOAD   = expected
STRESS = beyond
SPIKE  = sudden
SOAK   = sustained
```

---

55. Security Testing

Two terms are frequently confused:

Authentication

> **Who are you?**

Example:

```
Username + password
```

Authorization

> **What are you allowed to do?**

Example:

```
Normal employee cannot access Admin page.
```

Memory

> **AuthN = Name/identity**
>
> **AuthZ = Zone of permission**

---

56. Reliability and Recovery Testing

Reliability

Can the software continue operating correctly without failure?

Recovery

If failure occurs, can it recover safely?

Example:

A payment server crashes after receiving payment but before displaying confirmation.

Testing should ask:

- Was payment recorded once?
- Can service restart?
- Is data intact?
- Will retry accidentally charge twice?

Memory

> **Reliability prevents/limits failures**
>
> **Recovery handles what happens afterward**

---

57. Interoperability Testing

Check whether different systems work correctly together.

Example:

```
Mobile app
    ↓
REST API
    ↓
Payment gateway
    ↓
Bank
```

You test interfaces, formats and interactions between the components.

Memory

> **Interoperability = systems working together**

---

58. Web Application Testing

A web system can involve:

```
Browser
   ↓
Network
   ↓
Web server
   ↓
Application
   ↓
Database
```

Testing therefore shouldn’t focus only on what appears on screen.

---

59. Static vs Dynamic Web Pages

Static

Content is mostly predetermined.

Typical concern:

```
Links
layout
content
browser compatibility
```

Dynamic

Content depends on:

- user
- database
- session
- request
- server-side computation

Example:

```
Amazon-style shopping cart
Bank account
NPTEL login/dashboard
```

---

60. Client-Side vs Server-Side Testing

Suppose JavaScript prevents age `-5`.

You cannot conclude the system is secure just because the browser blocks it.

An attacker could send a request directly to the server containing:

```
age=-5
```

Therefore server-side behavior must also be tested.

Remember

> **Never assume browser validation alone protects the server.**

61. Bypass Testing

Instead of using the normal UI:

```
Browser → Form → Server
```

send crafted requests more directly to test what the server actually accepts.

Particularly pay attention to:

```
Request parameters
Cookies
Headers
Hidden form fields
```

Memory

> **Bypass the UI, test the boundary**

62. Mobile Application Testing

Mobile apps introduce special challenges:

```
Devices
OS versions
Screen sizes
Orientation
Permissions
Sensors
Network
Battery
Memory
Lifecycle
```

63. Emulator vs Real Device

Emulator

Useful for quickly testing many configurations.

Real device

Important when real hardware and platform behavior matter.

Memory

> **Emulator = breadth**
>
> **Real device = reality**

---

64. Mobile Lifecycle Testing

Imagine using an app:

```
Open app
↓
Receive call
↓
App backgrounds
↓
Return
↓
Rotate screen
↓
OS kills process
↓
Open again
```

The application should handle these lifecycle changes appropriately.

You should consider:

```
launch
background
foreground
rotation
process death
upgrade
reinstall
```

---

65. Mobile Network Testing

Mobile connectivity isn’t stable.

Test scenarios such as:

```
Wi-Fi → cellular
cellular → Wi-Fi
online → offline
weak network
high latency
request interrupted halfway
```

Example:

You click **Pay ₹500**.

Network drops.

You click again.

The system must handle the retry correctly and avoid unintended duplicate processing.

---

66. Mobile Permission Testing

Don’t test only:

```
User grants camera permission
```

Also test:

```
User denies it
User later grants it
User revokes it
```

The application shouldn’t simply crash when permission isn’t available.

□ MOST IMPORTANT NIGHT-BEFORE-EXAM MEMORY MAP

Try to reproduce this from memory:

```
SOFTWARE TESTING
│
├── INPUTS
│   ├── Complete = nothing missing
│   ├── Disjoint = no overlap
│   ├── ECC = each block
│   ├── ACoC = everything
│   ├── PWC = every pair
│   └── BCC = base + change one
│
├── GRAPHS
│   ├── NC = every node
│   ├── EC = every edge
│   ├── EPC = every 2-edge path
│   └── PPC = prime paths
│
├── DATA FLOW
│   ├── DEF
│   ├── USE
│   ├── c-use = computation
│   ├── p-use = predicate
│   └── All-defs → All-uses → All-du-paths
│
├── LOGIC
│   ├── PC = predicate T/F
│   ├── CC = each clause T/F
│   ├── CoC = 2^n
│   ├── GACC = minors may vary
│   ├── CACC = predicate changes
│   └── RACC = minors fixed
│
├── MUTATION
│   ├── Mutant
│   ├── Kill
│   ├── Equivalent mutant
│   └── Mutation score
│
├── OO
│   ├── inheritance
│   ├── overriding
│   ├── polymorphism
│   └── dynamic binding
│
├── REGRESSION
│   ├── Retest all
│   ├── Selection = choose
│   ├── Minimization = remove
│   └── Prioritization = reorder
│
└── NON-FUNCTIONAL
    ├── Load = expected
    ├── Stress = beyond
    ├── Spike = sudden
    └── Soak = sustained
```

If you remember only 8 things tomorrow

1. **RIPR:** Reach → Infect → Propagate → Reveal
2. **ISP:** Complete = nothing missing; Disjoint = no overlap
3. **ACoC:** multiply block counts
4. **Graph:** Node → Edge → Edge-Pair → Prime Path
5. **Data flow:** All-Defs → All-Uses → All-DU-Paths
6. **Logic:** GACC = general, CACC = predicate correlated, RACC = minors restricted/fixed
7. **Mutation:** surviving mutant may be weak testing **or equivalent mutant**
8. **Performance:** Load = expected, Stress = beyond, Spike = sudden, Soak = sustained

For your exam preparation, I would spend the most attention on **ISP calculations, graph/prime-path questions, data-flow DU-pairs, logic coverage (especially GACC/CACC/RACC), and mutation testing**, because those concepts require reasoning rather than simple memorization.